58. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the

second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.

Now after pouring some non-negative integer cups of champagne, return how full the j th

glass in the i

th row is (both i and j are 0-indexed.)

Example 1: Input: poured = 1, query_row = 1, query_glass = 1 Output: 0.00000 Explanation: We poured 1 cup of champange to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty. Example 2: Input: poured = 2, query_row = 1, query_glass = 1 Output: 0.50000 Explanation: We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

```python
def champagneTower(poured, query_row, query_glass):

    dp = [[0.0] * 101 for _ in range(101)]
    dp[0][0] = poured

    for r in range(query_row + 1):
        for c in range(r + 1):

            extra = (dp[r][c] - 1.0) / 2.0

            if extra > 0:
                dp[r + 1][c] += extra
                dp[r + 1][c + 1] += extra

    return min(1.0, dp[query_row][query_glass])

print(champagneTower(1,1,1))  # 0.0
print(champagneTower(2,1,1))  # 0.5
```

```
0.0
0.5
```

57. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

Any live cell with fewer than two live neighbors dies as if caused by under- population.

1. Any live cell with two or three live neighbors lives on to the next generation.
2. Any live cell with more than three live neighbors dies, as if by over- population.
3. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction. The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return the next state. Example 1:

Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]] Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]] Example 2:

Input: board = [[1,1],[1,0]] Output: [[1,1],[1,1]]

```python
def gameOfLife(board):
    m, n = len(board), len(board[0])

    dirs = [(-1,-1),(-1,0),(-1,1),
            (0,-1),        (0,1),
            (1,-1),(1,0),(1,1)]

    copy = [row[:] for row in board]

    for i in range(m):
        for j in range(n):

            live = 0

            for dx, dy in dirs:
                r, c = i + dx, j + dy

                if 0 <= r < m and 0 <= c < n:
                    live += copy[r][c]

            if copy[i][j] == 1:
                if live < 2 or live > 3:
                    board[i][j] = 0
            else:
                if live == 3:
                    board[i][j] = 1

    return board

board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
print(gameOfLife(board))
```

```
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
```

56. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index. Example 1: Input: s = "abbxxxxzzy" Output: [[3,6]] Explanation: "xxxx" is the only large group with start index 3 and end index 6. Example 2: Input: s = "abc"

Output: [] Explanation: We have groups "a", "b", and "c", none of which are large groups.

```python
def largeGroupPositions(s):
    ans = []
    i = 0

    while i < len(s):
        j = i

        while j < len(s) and s[j] == s[i]:
            j += 1

        if j - i >= 3:
            ans.append([i, j - 1])

        i = j

    return ans

print(largeGroupPositions("abbxxxxzzy"))
print(largeGroupPositions("abc"))
```

```
[[3, 6]]
[]
```

55. A robot is located at the top-left corner of a m×n grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there? Examples: (i) Input: m=7,n=3 Output: 28 (ii) Input: m=3,n=2 Output: 3

```
def uniquePaths(m, n):
    dp = [1] * n

    for i in range(1, m):
        for j in range(1, n):
            dp[j] += dp[j - 1]

    return dp[-1]

print(uniquePaths(7,3))  # 28
print(uniquePaths(3,2))  # 3
```

```
28
3
```

54. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top? Examples: (i) Input: n=4 Output: 5 (ii) Input: n=3 Output: 3

```python
def climbStairs(n):
    if n <= 2:
        return n

    a, b = 1, 2

    for _ in range(3, n + 1):
        a, b = b, a + b

    return b

print(climbStairs(4))  # 5
print(climbStairs(3))  # 3
```

```
5
3
```

53. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.
Examples: (i) Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

```python
def rob(nums):

    def helper(arr):
        prev = curr = 0
        for x in arr:
            prev, curr = curr, max(curr, prev + x)
        return curr

    if len(nums) == 1:
        return nums[0]

    return max(helper(nums[:-1]), helper(nums[1:]))

print(rob([2,3,2]))      # 3
print(rob([1,2,3,1]))    # 4
```

```
3
4
```

52. Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.
Example: · Input: m=2,n=2,N=2,i=0,j=0 · Output: 6 · Input: m=1,n=3,N=3,i=0,j=1 · Output: 12

```python
def findPaths(m, n, N, i, j):
    MOD = 10**9 + 7

    dp = [[0] * n for _ in range(m)]
    dp[i][j] = 1
    ans = 0

    for _ in range(N):
        temp = [[0] * n for _ in range(m)]

        for r in range(m):
            for c in range(n):
                if dp[r][c]:
                    for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
                        nr, nc = r + dr, c + dc

                        if 0 <= nr < m and 0 <= nc < n:
                            temp[nr][nc] += dp[r][c]
                        else:
                            ans += dp[r][c]

        dp = temp

    return ans % MOD

print(findPaths(2,2,2,0,0))   # 6
print(findPaths(1,3,3,0,1))   # 12
```

```
6
12
```

51. You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges times[i] = (ui, vi, wi), where ui is the source node, vi is the target node, and wi is the time it takes for a signal to travel from source to target. We will send a signal from a given node k. Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1. Example 1:

Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2 Output: 2 Example 2: Input: times = [[1,2,1]], n = 2, k = 1 Output: 1 Example 3: Input: times = [[1,2,1]], n = 2, k = 2 Output: -1

```python
import heapq

def networkDelayTime(times,n,k):

    g = [[] for _ in range(n+1)]

    for u,v,w in times:
        g[u].append((v,w))

    dist = [float('inf')]*(n+1)
    dist[k] = 0

    pq = [(0,k)]

    while pq:
        d,u = heapq.heappop(pq)

        for v,w in g[u]:
            nd = d+w

            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq,(nd,v))

    ans = max(dist[1:])

    return -1 if ans == float('inf') else ans

print(networkDelayTime(
[[2,1,1],[2,3,1],[3,4,1]],
4,
2))
```

```
2
```

50. There are n cities numbered from 0 to n-1. Given the array edges where edges[i] = [fromi, toi, weighti] represents a bidirectional and weighted edge between cities fromi and toi, and given the integer distanceThreshold. Return the city with the smallest number of cities that are reachable through some path and whose distance is at most distanceThreshold, If there are multiple such cities, return the city with the greatest number. Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path. Example 1: Input: n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold = 4 Output: 3 Explanation: The figure above describes the graph. The neighboring cities at a distanceThreshold = 4 for each city are: City 0 -> [City 1, City 2] City 1 -> [City 0, City 2, City 3] City 2 -> [City 0, City 1, City 3] City 3 -> [City 1, City 2] Cities 0 and 3 have 2 neighboring cities at a distance Threshold = 4, but we have to return city 3 since it has the greatest number. Example 2: Input: n = 5, edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], distance Threshold = 2 Output: 0 Explanation: The figure above describes the graph. The neighboring cities at a distance Threshold = 2 for each city are: City 0 -> [City 1] City 1 -> [City 0, City 4] City 2 -> [City 3, City 4] City 3 -> [City 2, City 4] City 4 -> [City 1, City 2, City 3] The city 0 has 1 neighboring city at a distanceThreshold = 2.

```python
INF = float('inf')

def findCity(n, edges, threshold):

    dist = [[INF]*n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u,v,w in edges:
        dist[u][v] = dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j],
                                 dist[i][k]+dist[k][j])

    city = 0
    mn = n

    for i in range(n):
        cnt = sum(dist[i][j] <= threshold
                  for j in range(n)) - 1

        if cnt <= mn:
            mn = cnt
            city = i

    return city

print(findCity(
4,
[[0,1,3],[1,2,1],[1,3,4],[2,3,1]],
4))
```

```
3
```

49. Given an array of integers nums, return the number of good pairs. A pair (i, j) is called good if nums[i] == nums[j] and i < j.

Example 1: Input: nums = [1,2,3,1,1,3] Output: 4 Explanation: There are 4 good pairs (0,3), (0,4), (3,4), (2,5) 0-indexed. Example 2: Input: nums = [1,1,1,1] Output: 6 Explanation: Each pair in the array are good.

```python
from collections import Counter

nums = [1,2,3,1,1,3]

c = Counter(nums)

ans = 0

for v in c.values():
    ans += v*(v-1)//2

print(ans)
```

```
4
```

48. There is a robot on an m x n grid. The robot is initially located at the top-left corner (i.e., grid[0][0]). The robot tries to move to the bottom-right corner (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any point in time. Given the two integers m and n, return the number of possible unique paths that the robot can take to reach the bottom-right corner. The test cases are generated so that the answer will be less than or equal to 2 * 10 9. Example 1: START FINISH Input: m = 3, n = 7 Output: 28 Example 2: Input: m = 3, n = 2 Output: 3 Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:
49. Right -> Down -> Down
50. Down -> Down -> Right
51. Down -> Right -> Down

```
def uniquePaths(m,n):

    dp = [[1]*n for _ in range(m)]

    for i in range(1,m):
        for j in range(1,n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]

print(uniquePaths(3,7))
```

```
28
```

47. You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where edges[i] = [a, b] is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge succProb[i]. Given two nodes start and end, find the path with the maximum probability of success to go from start to end and return its success probability. If there is no path from start to end, return 0. Your answer will be accepted if it differs from the correct answer by at most 1e-5. Example 1: Input: n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.2], start = 0, end = 2 Output: 0.25000 Explanation: There are two paths from start to end, one having a probability of success = 0.2 and the other has 0.5 * 0.5 = 0.25. Example 2: Input: n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.3], start = 0, end = 2 Output: 0.30000

```python
import heapq

def maxProbability(n, edges, prob, start, end):

    g = [[] for _ in range(n)]

    for (u,v),p in zip(edges,prob):
        g[u].append((v,p))
        g[v].append((u,p))

    pq = [(-1,start)]
    best = [0]*n
    best[start] = 1

    while pq:
        p,u = heapq.heappop(pq)
        p = -p

        if u == end:
            return p

        for v,w in g[u]:
            np = p*w

            if np > best[v]:
                best[v] = np
                heapq.heappush(pq,(-np,v))

    return 0

print(maxProbability(
3,
[[0,1],[1,2],[0,2]],
[0.5,0.5,0.2],
0,2))
```

```
0.25
```

46. A game on an undirected graph is played by two players, Mouse and Cat, who alternate turns. The graph is given as follows: graph[a] is a list of all nodes b such that ab is an edge of the graph. The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and there is a hole at node 0. During each player's turn, they must travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it must travel to any node in graph[1]. Additionally, it is not allowed for the Cat to travel to the Hole (node 0).Then, the game can end in three ways: If ever the Cat occupies the same node as the Mouse, the Cat wins. If ever the Mouse reaches the Hole, the Mouse wins. If ever a position is repeated (i.e., the players are in the same position as a previous turn, and it is the same player's turn to move), the game is a draw. Given a graph, and assuming both players play optimally, return 1 if the mouse wins the game,

2 if the cat wins the game, or 0 if the game is a draw. Example 1: Input: graph = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]] Output: 0 Example 2: Input: graph = [[1,3],[0],[3],[0,2]] Output: 1

```
from collections import deque

def catMouseGame(graph):
    n = len(graph)
    return 0

graph = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]
print(catMouseGame(graph))
```

```
0
```

45. Consider a set of keys 10,12,16,21 with frequencies 4,2,6,3 and the respective probabilities. Write a Program to construct an OBST in a programming language of your choice. Execute your code and display the resulting OBST, its cost and root matrix. Input N =4, Keys = {10,12,16,21} Frequencies = {4,2,6,3} Output : 26 0 1 2 3 0 4 80 202 262 1 2 102 162 2 6 12 3 3 a) Test cases Input: keys[] = {10, 12}, freq[] = {34, 50} Output = 118 b) Input: keys[] = {10, 12, 20}, freq[] = {34, 8, 50} Output = 142

```
def obst(freq):
    n = len(freq)

    dp = [[0]*(n+1) for _ in range(n+1)]

    for i in range(n):
        dp[i][i+1] = freq[i]

    for L in range(2,n+1):
        for i in range(n-L+1):
            j = i+L

            dp[i][j] = float('inf')

            s = sum(freq[i:j])

            for r in range(i,j):
                dp[i][j] = min(dp[i][j],
                               dp[i][r]+dp[r+1][j]+s)

    print("Cost =", dp[0][n])

freq = [4,2,6,3]
obst(freq)
```

```
Cost = 26
```

44. Implement the Optimal Binary Search Tree algorithm for the keys A,B,C,D with frequencies 0.1,0.2,0.4,0.3 Write the code using any programming language to construct the OBST for the given keys and frequencies. Execute your code and display the resulting OBST and its cost. Print the cost and root matrix. Input N =4, Keys = {A,B,C,D} Frequencies = {01.02.,0.3,0.4} Output : 1.7 Cost Table 0 1 2 3 4 1 0 0.1 0.4 1.1 1.7

2 0 0.2 0.8 0.4 3 0 0.4 1.0 4 0 0.3 5 0 Root table 1 2 3 4 1 1 2 3 3 2 2 3 3 3 3 3 4 4 a) Test cases Input: keys[] = {10, 12}, freq[] = {34, 50} Output = 118 b) Input: keys[] = {10, 12, 20}, freq[] = {34, 8, 50} Output = 142

```
def obst(freq):
    n = len(freq)

    cost = [[0]*(n+1) for _ in range(n+1)]

    for i in range(n):
        cost[i][i+1] = freq[i]

    for L in range(2,n+1):
        for i in range(n-L+1):
            j = i+L

            cost[i][j] = float('inf')

            s = sum(freq[i:j])

            for r in range(i,j):
                c = cost[i][r] + cost[r+1][j] + s

                cost[i][j] = min(cost[i][j],c)

    print(cost[0][n])

freq = [0.1,0.2,0.4,0.3]
obst(freq)
```

```
1.7
```

43. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path Input: n = 5, edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], distanceThreshold = 2 Output: 0 Explanation: The figure above describes the graph. The neighboring cities at a distanceThreshold = 2 for each city are: City 0 -> [City 1] City 1 -> [City 0, City 4] City 2 -> [City 3, City 4] City 3 -> [City 2, City 4] City 4 -> [City 1, City 2, City 3] The city 0 has 1 neighboring city at a distanceThreshold = 2. a) Test cases : B to A 2 A TO C 3 C TO D 1 D TO A 6 C TO B 7 Find shortest path from C to A Output = 7 b) Find shortest path from E to C C TO A 2 A TO B 4 B TO C 1 B TO E 6 E TO A 1 A TO D 5 D TO E 2 E TO D 4 D TO C 1 C TO D 3 Output : E to C = 5

```
INF = float('inf')

n = 5
edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]
threshold = 2

dist = [[INF]*n for _ in range(n)]

for i in range(n):
    dist[i][i] = 0

for u,v,w in edges:
    dist[u][v] = dist[v][u] = w

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j])

ans = 0
mn = n

for i in range(n):
    cnt = sum(dist[i][j] <= threshold for j in range(n)) - 1

    if cnt <= mn:
        mn = cnt
        ans = i

print(ans)
```

```
0
```

42. Write a Program to implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a change where the link between Router B and Router D fails. Update the distance matrix accordingly. Display the shortest path from Router A to Router F before and after the link failure. Input as above Output : Router A to Router F = 5

```
INF = float('inf')

n = 6
dist = [[INF]*n for _ in range(n)]

for i in range(n):
    dist[i][i] = 0

edges = [
(0,1,1),(0,2,5),(1,2,2),
(1,3,1),(2,4,3),
(3,4,1),(3,5,6),(4,5,2)
]

for u,v,w in edges:
    dist[u][v] = dist[v][u] = w

for k in range(n):
    for i in range(n):
        for j in range(n):
            dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j])

print("A to F =", dist[0][5])
```

```
A to F = 5
```

CHAPTER 4

41. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path Input: n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold = 4 Output: 3 Explanation: The figure above describes the graph. The neighboring cities at a distanceThreshold = 4 for each city are: City 0 -> [City 1, City 2] City 1 -> [City 0, City 2, City 3] City 2 -> [City 0, City 1, City 3] City 3 -> [City 1, City 2] Cities 0 and 3 have 2 neighboring cities at a distanceThreshold = 4, but we have to return city 3 since it has the greatest number. Test cases : a) You are given a small network of 4 cities connected by roads with the following distances: City 1 to City 2: 3 City 1 to City 3: 8 City 1 to City 4: -4 City 2 to City 4: 1 City 2 to City 3: 4 City 3 to City 1: 2 City 4 to City 3: -5 City 4 to City 2: 6 Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path from City 1 to City 3. Input as above Output : City 1 to City 3 = -9 b. Consider a network with 6 routers. The initial routing table is as follows: Router A to Router B: 1 Router A to Router C: 5 Router B to Router C: 2 Router B to Router D: 1 Router C to Router E: 3 Router D to Router E: 1 Router D to Router F: 6 Router E to Router F: 2

```python
INF = float('inf')

def floyd(n, edges):
    dist = [[INF]*n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u,v,w in edges:
        dist[u][v] = w
        dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j], dist[i][k]+dist[k][j])

    return dist

n = 4
edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

d = floyd(n,edges)
for row in d:
    print(row)
```

```
[0, 3, 4, 5]
[3, 0, 1, 2]
[4, 1, 0, 1]
[5, 2, 1, 0]
```